QUESTION 1:

[6 POINTS]

Multiple choice questions. Select the correct answer.

A. [1 POINT] What is an object in Java?

a)	A primitive data type.	b)	A reference for class.
c)	A collection of methods only.	d)	An instance of a class.

B. [1 POINT] Which access modifier allows members to be accessed by any class in any package?

a)	public.
b)	private.
c)	protected.
d)	default.

C. [1 POINT] What does the == operator compare when used with reference variables in Java?

a)	The actual content of the objects.	b)	The size of the objects.
c)	The memory addresses of the objects.	d)	The class types of objects.

D. [1 POINT] What happens when you assign one reference variable to another?

a)	The original object is destroyed.	b)	A new object is created.
c)	Both variables refer to the same object in memory.	d)	The compiler will throw an error.

E. [1 POINT] non-static variables can't be referenced from a static method in Java.

a)	True.	b)	False.

F. [1 POINT] What is the main purpose of encapsulation in Java?

a)	To hide and protect the data.	b)	To increase the performance of the program.
c)	To define new classes.	d)	To implement inheritance.

Scanned with CamScanner

QUESTION 2: Evaluate the following Java codes and show the exact output. **[4.5 POINTS]**

A. [2 points]

```
public class TestClass {
   public static void main(String[] args) {

      String str = "ASU University is an outstanding University";
      String Str1 = str.toLowerCase();
      int f1 = Str1.lastIndexOf('u', 20); 4
      int f2 = Str1.indexOf('u', 10); 22
      String substring = str.substring(f1, f2); university is an ou
      System.out.println(f1); 4
      System.out.println(f2); 22
      System.out.println(substring);
   }
}
```

4
22
university is an o

2

B. [2.5 points]

```
public class Book {
  String title;
   double price;
   static int x=14;
   public Book(){
      this("unknown",0);
      System.out.println(1);
      x/=2; 7÷2=3
   }
   public Book(String title,double price )
   {
      this.title=title;
      this.price=price;
      System.out.println(2);
      x/=2; 14÷2=7
   }}
public class Test {
   public static void main(String[] args) {
      Book b1=new Book();
      System.out.println(b1.title);
      System.out.println(Book.x);
     System.out.println(b1.price);
   }
}
```

2
1
unknown
3
0

2
1
unknown
3
0

2.5

Scanned with CamScanner

QUESTION 3: [4.5 POINTS]

Write a **static method** named **isPrefixSuffixSame(String str)** that returns **true** if the **first two and last two characters** of the string are the same. If they are not the same, return **false**. The string **must** be at least 2 characters long (Return false if string length is less than 2). if (charAt(0)&&(charAt(1))==charAt(size

size ≥ 2 str.length()= size; String last= charAt

Sample 1:	Sample 2:	Sample 3:
Input: edited Prefix: "ed" Suffix: "ed" Output: true	Input: hello Prefix: "he" Suffix: "lo" Output: false	Input: h Output: false

1

```
public static boolean isPrefixSuffixSame ( String str ){
  String first = str.charAt(0);
  String sec = str.charAt(1);
  String prefix= first.concat ( sec);
  int size = str.length();
  String last = str.charAt ( size-1);
  String beforeLast = str.charAt ( size-2);
  String suffix= beforeLast.concat ( last);
  if ( size >= 2) {
      if ( prefix.equals (suffix)) {
          return true;  }
      else {
          return false; }
  }
  else {
    return false; }
}
```

0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5

```
public String toString ( ) {
    return "Employee ID: " + empId + "\n" + "Employee name: " +
    empName + "\n" + "Employee salary: " + salary + "\n" +
    "Contract type: " + myContract.getContractType() + "\n"
    + "Start date: " + myContract.getStartDate() + "\n"
    + "End date: " + myContract.getEndDate() ; }
}
```

2.25

Scanned with CamScanner

QUESTION 4: [15 POINTS]

Contract part of emp.

Design an **Employee Contract Management System** by creating two classes: **Employee** and **Contract**. The system should allow each employee to be associated with a contract, track their personal details and salary, and display complete information including their contract data.

In this system, an **Employee owns a Contract**. This means that the contract is tightly bound to the employee and does not exist independently. When an employee is created, their contract is also created and assigned to them as part of the employee object. You are required to implement the following classes according to the specifications provided below.
Note: You are not required to write the `main` method.

A. **[4.5 points]** The **Contract** class should represent an employment contract with the following

part of

- private instance variables:
  - **contractType** of type String – represents the type of contract (e.g., "Full-Time", "Part-Time", "Temporary").
  - **startDate** of type String – represents the start date of the contract (e.g., "2025-01-01").
  - **endDate** of type String – represents the end date of the contract (e.g., "2025-12-31").

- The **Contract** class must include:
  - A **parameterized constructor** to initialize contractType, startDate, and endDate.
  - A **getter method** to return the contractType.
  - A **getter method** to return the startDate.
  - A **getter method** to return the endDate.

  no setter.

B. **[10.5 points]** The **Employee** class should represent an employee and contain the following:

(~~myContract.getContractType()~~)

- private instance variables:
  - **empId** of type String – represents the employee's unique ID.
  - **empName** of type String – represents the employee's name.
  - **salary** of type double – represents the employee's current monthly salary.
  - **contract** of type Contract – represents the contract associated with the employee.

- The **Employee** class must include:
  - A **parameterized constructor** with six parameters to initialize empId, empName, salary, and the contract details (contractType, startDate, and endDate).
  - An **increaseSalaryBasedOnContract() method** that increases the employee's salary based on the contract type. This method should **not return any value** and should directly update the salary field using the following rules:
    - **Full-Time**: Increase salary by 10%
    - **Part-Time**: Increase salary by 5%
    - **Temporary**: Increase salary by 3%

    salary += (salary * 10%)
    if (myContract.getContractType().equals(

  - A **toString() method** that returns a formatted string containing the employee's details, including their contract information.

Scanned with CamScanner

```
public class Contract {
    private String contractType;
    private String startDate;
    private String endDate;

    public Contract (String contractType, String startDate, String endDate){
     this.contractType = contractType;
     this.startDate = startDate;
     this.endDate = endDate;
     }

    public String getContractType( ){
        return contractType; }
    public String getStartDate( ) {
        return startDate; }
    public String getEndDate ( ){
        return endDate; }
}
```

1

2

1.5

```
public class Employee {
  private String empId;
  private String empName;
  private double salary;
  private Contract myContract;
  public Employee (String empId, String empName, double salary, String c, String s, String e){
  this.empId = empId;
  this.empName = empName;
  this.salary = salary;
  myContract = new Contract (c, s, e); }
  public void increaseSalaryBasedOnContract ( ){
   if ( myContract.getContractType.equals ("Full-Time"){
        Salary = Salary + ( salary * 0.1 ); }
   else if (myContract.getContractType.equals ("Part-Time"){
        Salary = Salary + ( salary * 0.05 ); }
   else if (myContract.getContractType.equals ("Temporary"){
        Salary = Salary + ( salary * 0.03 ); }
   else { Salary += 0; }
  }
```

1

0.5

2.5

1.25

1

1

1

to string next page